

TECHNISCHE
UNIVERSITÄT
WIEN

Introduction to C

191.002 Operating Systems VU
2026W

Luca Di Stefano, Jaroslav Klapalek, Joel Klimont, Felix Resch,
Peter Puschner
2026-10-09

Part I

Overview

- 1969: UNIX project starts at Bell Labs
- Initial plan: write the OS in assembly language (for PDP machines)
- Not portable, time-consuming, error-prone. Needed an alternative
- C was developed, as successor to the languages B and ALGOL (ALGOrithmic Language)

```
" copy zeroes (clear memory)
" call:
"   jms copyz; pointer; count
copyz: 0
   -1
   tad copyz i  " get call PC
   dac 8        " save in index (pre-increments)
   isz copyz    " skip pointer
   -1
   tad copyz i          " get count-1
   cma                  " get negative count
   dac 9f+t             " save in t0
   isz copyz            " skip count
1:
   dzm 8 i              " zero word
   isz 9f+t             " done?
   jmp 1b               "  no: loop
   jmp copyz i          " return
t = t+1
```

Source: PDP-7 Unix Source code[1]

[1] https://www.tuhs.org/cgi-bin/utree.pl?file=PDP7-Unix/sys/s4.s

Standards

1978: "K&R C"

- *De facto* standard, described in the Kernighan & Ritchie book

1989: C89 aka ANSI-C ⟵ most portable version

- First actual standardization of the language

1999 C99 ⟵ "our" version of choice for this course

- Several improvements to syntax, new features
- Not (fully) supported by all compilers

2011–now: C11, C17, C23

- Some compilers implement (non-standard) extensions of C
- E.g., "GNU C" in GCC (this is how Linux is written)
- We use GCC but we will force it to use C99

Facts about C

Imperative: State (variables) and statements (instructions) to change it

Procedural: Statements are grouped into procedures (functions) that may take arguments and may return values

Statically Typed: Each variable has a type (e.g., integer, floating point, character) that does not change. Functions also have a return type

Compiled: Programs are translated into machine code by a compiler

Preprocessed: The standard defines a preprocessor that alters the source code before compilation. Used e.g., to include other code and define macros (textual replacements)

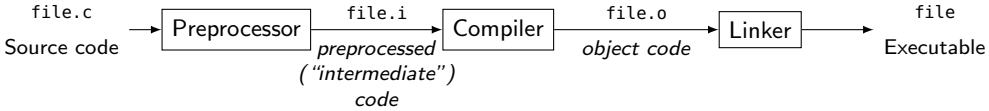
- `#include <stdio.h>`: Loads the contents of `stdio.h`, which is part of the standard library (this is an instruction to the preprocessor)
- `int main(void)`: Function named `main` that takes no arguments (`void`) and returns an integer (`int`)
- `printf`: Function defined in `stdio.h` that prints, i.e., sends bytes to stdout
- The value returned by `main` is the exit code of the program

```c
#include <stdio.h>

int main(void){
  printf("Hello, C World\n");
  return 0;
}
```

Compilation: Translate source code into object code (= assembly instructions)
Linking: Combine object files and libraries into an executable

Full process, using GCC: `$ gcc -o prog prog.c`



Individual steps:

```
$ gcc -E prog.c      # preprocess only
$ gcc -c hello.c     # compile only (get object file)
```

Part II

Introduction

Comments


```c
/* I am a comment in C89 */

/* I can spread
    over multiple
    lines */

// I am a comment in C99. A C89 compiler will not accept me!
// I end at the end of the line

// Comments usually improve readability
// However, use them for useful info/context!
// E.g., is the comment below useful?

// Sets x to the sum of y and z
x = y + z;
```

Variables

- A variable is a location in memory with a name and a type
- The amount of memory used depends on the type
- Variables are declared and/or defined and/or initialized
  (Please do initialize your variables!)
- Variables defined outside of any function are called global variables

```
1  int i;        // Integer named i, declared and defined (not initialized)
2  int j = 23;   // j declared, defined, AND initialized to 23
3
4  i = 42;       // Assignment to already-defined variable i
5
6  extern int k; // declared, but defined somewhere else
```

The linker will find where k is defined at link time.

Examples


```c
int i; /* declaration and definition of a single integer variable */

int i, j, k; // declare and define multiple integers at once

int i, j = 23, k = 42; /* same, but some variables are initialized */

int i, char b; // incorrect syntax

int i; char b; /* correct, declares and defines an int and a char
                  (; -> separate statements, may be on the same line) */

int i = 4; char b = 'A'; // same with initializations
```

(But please keep your code tidy and readable!)

Scopes

The scope of a variable is the region of the code where the variable can be accessed.

- Usually the block (i.e., code between {}) where the variable is defined
- Global variables have global scope, i.e., they can be accessed from anywhere.
- You cannot use the same variable name twice in the same scope
- But you can reuse a name in a nested scope ("shadowing")

```c
#include <stdio.h>

int x = 120;
int main(void) {
  int x = 0;
  printf("Hello World\n");
  return x;
}
```

Declarations before C99

- With C99 we can put declarations anywhere
- Before, variables had to be declared at the beginning of a block
- Tip: put declarations close to where you need the variable

```c
#include <stdio.h>

int main(void){
  printf("Hello World\n");
  int j = 42; /* not at the beginning of the block */
  printf("The value of j is %d\n", j);
  return 0;
}
```

Byte in C99

> *A byte is composed of a contiguous sequence of bits, the number of which is implementation-defined.*
>
> *ISO C99, 3.6*

> *implementation-defined: unspecified behavior where each implementation documents how the choice is made*
>
> *ISO C99, 3.4.1*

Not necessarily 8 bits! E.g., some PDP machine had 16-bit bytes.

But most likely your machine uses 8 bits :)

Integer Types[1]

char:	1 byte. Often used to store *characters*
short int:	min. 2 bytes
int:	4 or 8 bytes (usually 4)
long int:	min. 4 bytes
long long int:	min. 8 bytes (C99)

All types have signed and unsigned variants. Unsigned can only store values ≥ 0.

Signed by default: write `unsigned <type>` for unsigned
But `char` default signedness is implementation-defined

Actual sizes are available in `<limits.h>`
C99 introduced `<stdint.h>` with fixed sizes: `uint32_t`, `int8_t`, ...

[1] See also `https://en.wikipedia.org/wiki/C_data_types`

> *The range of nonnegative values of a signed integer type is a subrange of the corresponding unsigned integer type, and the representation of the same value in each type is the same.*
>
> *ISO C99, 6.2.5*

Some typical (but not standardised) ranges:

Type	Signed min	Signed max	Unsigned min	Unsigned max
char	-128	127	0	255
short int	-32.768	32.767	0	65.535
long int	-2.147.483.648	2.147.483.647	0	4.294.967.295

Real Floating Types

`float`, `double`, `long double`: can store (a subset of) real numbers

> *The set of values of the type float is a subset of the set of values of the type double; the set of values of the type double is a subset of the set of values of the type long double.*
>
> *ISO C99, 6.2.5 (very vague!)*

Usually float = 32-bit, double = 64-bit, and long double = 80-bit (or 128-bit)

The operator `sizeof` may also be used to get the memory needed by a type

```
1 int i;
2 printf("%lu byte(s)\n", sizeof i);
3 printf("%lu byte(s)\n", sizeof (int));
```

Strange syntax in line 2?

Remember, sizeof is an operator

Part III

Operators

Arithmetic Operators

- Binary operators: take two operands, e.g., `+`, `-`, `*`, `/`, `%`
- Unary operators: take one operand, e.g., `++`, `--`, `sizeof`
- `x++` computes $x + 1$, assigns it to x, and returns the old value of x
- `++x` computes $x + 1$, assigns it to x, and returns the new value of x

One ternary operator: `expr1 ? expr2 : expr3`
$\Rightarrow$ "if `expr1` is true, then `expr2`, else `expr3`"

Truthiness

In C, 0 is false, everything else is true

Logical Operators

- ! (Logical NOT). E.g., `!0` is 1, `!42` is 0
- Comparison operators: `<`, `>`, `<=`, `>=`, `==`, `!=`
- `&&` (AND), `||` (OR)

Short-circuit evaluation: when `e1 && e2` is evaluated:

1. Evaluate `e1`
2. If `e1` is false (0), return 0
3. Otherwise, evaluate `e2` and return its value.

Mirrored for `e1 || e2`: `e2` evaluated only if `e1` is false

Operators that work bit by bit

x	1	0	0	1	1	0	1	1
y	0	0	0	1	0	0	0	1
x & y	0	0	0	1	0	0	0	1
x | y	1	0	0	1	1	0	1	1
x ^ y	1	0	0	0	1	0	1	0
x << n	0	0	1	1	0	1	1	0
x >> n	0	1	0	0	1	1	0	1
~x	0	1	1	0	0	1	0	0

x	
y	
x & y	and
x | y	or
x ^ y	xor (exclusive or: like or, but 1^1 = 0)
x << n	shift left by *n* positions (here n=1)
x >> n	shift right by *n* positions (here n=1)
~x	bit-wise complement (= each 0 replaced by 1 and vice versa)

Notes

- For this value of x, !x is simply 0
- & and | do not short-circuit

Part IV

Functions

Definition of Functions


```
type name(type1 param1, type2 param2, ...) {
   /* code */
}
```

- Increase the readability, re-usability and maintainability
- Need to be declared before they can be used

```
int add(int a, int b) {
   return a + b;
}

int main(void) {
   int i;
   i = add(2, 3);
   /* i == 5 */
   return 0;
}
```

Prototypes

- Like variable, declaration and definition are differentiated
- A prototype represents a declaration and ends with an ';'

```
1  /* Prototype */
2  int add(int a, int b);
3  /* int add(int x, int v); also okay */
4  /* int add(int, int); also okay */
5  /* int add(double, int); wrong, because int is used later */
6
7  int main(void) {
8     int i;
9     i = add(2, 3); /* i == 5 */
10    return 0;
11 }
12 /* now add can be defined even "after" it has been called */
13 int add(int a, int b) {
14    return a + b;
15 }
```

Global vs Local

- Local variables are invisible outside their block
- Global variables (declared outside of functions) are accessible until the program ends
- Local variables mask global variables
- Local variables have a random value at definition, unless initialized
- Global variables are initialized with 0 by default

Part V

Control Structures

if


```
1  if (expression)
2     statement
3  else if (expression)
4     statement
5  else if (expression)
6     statement
7  /* . . . */
8  else
9     statement
```

Evaluate expression at line 1. If true:

1. Execute statement at line 2
2. Jump to end of the entire block (past line 9)

If false:

1. Jump to else if at line 3
2. Evaluate expression
3. . . .

Tip: always embrace ({}) the `statement`, even if just 1 line

```c
switch (expression) {
  case const_expr: statements
  case const_expr: statements
  /* . . . */
  default: statements
}
```

```c
/* Example */
char input;
// input = ...;
switch (input) {
  case 'a':
  case 'A':
    printf("a or A\n");
    break;
  default:
    printf("Some other character\n");
    break;
}
```

- Evaluates expression and compare it to each case. Takes the first match
- Only constant values can be used for equality checks
- A case should end with `break`, otherwise the next case(s) will execute!
- Always provide a `default` case

while/do-while; continue; break


```
1  while (expression)
2     statement
```

```
1  do
2     statement
3  while (expression);
```

- Both used for loops: repeats `statement` as long as `expression` is true
- While may never execute `statement` (if `expression` is false at the beginning)
- Do-while executes `statement` at least once
- `continue` starts the next iteration of the most inner loop
- `break` exits the most inner loop

for


```
1 for (expr1; expr2; expr3)
2    statement
```

```
1 // Typical example
2 for (int i = 0; i < x; ++i) { ... }
```

- The expressions are not mandatory: e.g., `for(;;)` is an infinite loop
- `expr1`: Performed before the first iteration (e.g., initialize counter)
- `expr2`: Used to check whether the loop should continue
- `expr3`: Performed after each iteration (e.g., increment counter)

Behaviour on continue/break

- `continue`: `expr3` is executed, then `expr2` is checked;
  if true, a new iteration starts
- `break`: Exits immediately

```
1  int x = 0;
2  incr: ++x;
3  if (x < 5) {
4    goto incr;
5  }
```

- Any statement/block can be labeled with an identifier
- `goto` jumps to a label in the same function
- Preferably to be avoided!!!
- "Considered harmful"[2]

[2] Edsger W. Dijkstra, https://homepages.cwi.nl/~storm/teaching/reader/Dijkstra68.pdf

goto fail; goto fail;[3]

Apple's libsecurity_ssl, `sslKeyExchange.c`:

```
1  SSLVerifySignedServerKeyExchange(..)
2      if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
3        goto fail;
4        goto fail;
5      if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
6        goto fail;
7    fail:
8      SSLFreeBuffer(&signedHashes);
9      SSLFreeBuffer(&hashCtx);
10     return err;
```

- http://opensource.apple.com/source/Security/Security-55471/libsecurity_ssl/lib/sslKeyExchange.c
- http://www.theregister.co.uk/2014/02/25/apple_mac_os_x_10_9_2_ssl/

[3] Thanks to Roland Kammerer for this example!

Part VI

Modifiers

- Declares variables which are defined in another file

inc.c

```
1 int g_variable = 1;
2 [..]
3   g_variable++;
4 [..]
```

dec.c

```
1 extern int g_variable;
2 [..]
3   g_variable--;
4 [..]
```

const


```
1 const int i = 23; // C constant
```

- Defines a read-only variable `i` and sets it to 23
- Attempts to change the value of `i`: compiler error

Alternative:

```
1 #define MYCONST 23
```

- Command to the preprocessor, not a variable
- `MYCONST` is replaced with 23 during preprocessing
  (will not be seen by the compiler at all!)

static

- `static` gives a fixed memory space to the variable, its state remains
- A static variable cannot be accessed from an outside block or file

```c
#include <stdio.h>

void foo(void) {
  static int i = 23;
  printf("%d, ", i);
  i = i + 1;
}
```

```c
int main(void)
{
  foo();
  foo();
  foo();
  return 0;
}
```

```
$ 23, 24, 25,
```

volatile

- Variable can be changed by the environment outside of the program
- Important for hardware programming (e.g., variable mapped to a sensor) and some interactions with the OS

```
1 // Imagine this is global and updated 'elsewhere'
2 // every time a key is pressed on the keyboard
3 volatile char keyPressed;
4
5 long count = 0;
6 while (keyPressed != 'x') { ++count; }
```

Without `volatile`:

- Global variable = value is initialized to 0
- Compiler sees that `keyPressed` is not changed in the loop
- So, always different from 'x' $\Rightarrow$ Compiler optimizes into an infinite loop

Part VII

Arrays

One Dimensional Arrays

- An array is a sequence of variables of the same type, of fixed length
- These variables are stored in consecutive memory locations
- You cannot use a variable as the size! (Only constant expressions)

```
1 int myarray[8];
```

- `myarray` stores 8 integer variables
- `myarray[0]`, `myarray[1]`, ..., `myarray[7]`
- `myarray[8]` out-of-bounds

Multi Dimensional Arrays

- Arrays can have multiple dimensions
- In C it is basically "syntactic sugar"

```
1 int myarray[2][3];
2 int myarray2[2][3][4];
```

Initialization (C99)


```c
/* This is nicer */
int myarr[2][3]= {
                 {1,2,3},
                 {4,5,6},
                 };
/* This also works */
int myarr2[2][3] = {1,2,3,4,5,6};

/* if the whole array is initialized
you do not need to declare the size */
int myarr3[] = {1, 2, 3, 4};

/* if not all elements are initialized,
the rest is set to 0 */
int myarr4[100] = {1, 2, 3};
```

Strings

- C strings are arrays of characters: `char[]`
- Double quotes for strings, single for individual characters
- Strings are terminated with `'\0'` (“null character”)

```
1  char string[] = "hello, world";                /* \0 is added automatically */
2  char s[6];
3  s[0] = 'h'; s[1] = 'e'; s[2] = 'l';
4  s[3] = 'l'; s[4] = 'o'; s[5] = '\0';           /* must add \0 manually */
5  char str[] = {'f','o','o','b','a','r','\0'};   /* must add \0 manually */
6
7  printf("%s\n", s); /* prints "hello" */
8  s[3] = '\0';
9  printf("%s\n", s); /* prints "hel": function stops at \0 */
```

Part VIII

Pointers

Pointers

- A pointer is a variable
- Stores the address (memory location) of another variable
- This allows to access the value of a variable without using its name
- Of course you can have pointers to pointers, etc. (!)

Widely used in C:

- Arrays accesses are done via pointers
- When you ask for memory from the system, you get a pointer
- Functions can take a pointer to a location they should work on
- ...

Declaration

- `type *name`: declares a pointer `name` to a variable of type `type`
- The memory reserved for `name` does not have the size of `type`
- Rather, it has the size of a memory address (usually 4 or 8 bytes)

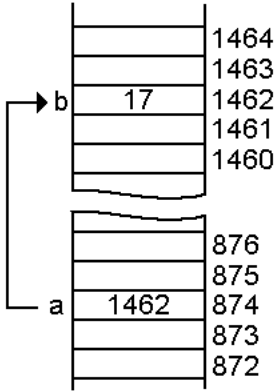
```
1 int *p; /* I'm a pointer */
2 int* q; /* Me too */
3 int* a, b; /* a is a pointer, but b is NOT a pointer */
4 int *a, b; /* a yep, b nope */
5 int *a, *b; /* a and b are pointers */
```

Rule of thumb: Declare one pointer per line

- Dereferencing operator *: Gets value pointed by the pointer
- Address operator &: Gets address of a variable

```
1    int *a;
2    int b = 17;
3    a = &b;
```

$$
\begin{aligned}
b &= 17 \\
\&b &= 1462 \\
a &= 1462 \\
*a &= 17 \\
\&a &= 874
\end{aligned}
$$



Simple Pointer Arithmetic


```c
int ar[5] = {1, 2, 3, 4, 5};
int *p;

p = &ar[0];
/* or */
p = ar; /* here, ar is treated as the address of ar[0] */

printf("%d\n", *p); /* 1 */
*p += 22;
printf("%d\n", ar[0]); /* 23 */
p += 1; /* pointer points to the next element */
printf("%d\n", *p); /* 2 */
```

`p += 1` does not always add 1 to the address

It adds n big enough to point to the next element of the pointer's type.

`char*` $\rightarrow n = 1$, `int*` $\rightarrow n = 4$ (usually), ...

Can get risky (more on this later in the course)

Part IX

Preprocessor

Preprocessor

- The preprocessor is called before the compiler runs
- Is doing simple replacements in the source code (case sensitive)
- Resulting source code can be viewed by running `gcc -E`

Some tasks of a preprocessor:

- Combining lines that are split by `'\'`
- Replacing macros and copying files (`#include`) into the source code

Replacing Constants


```
1 #define ANSWER (42) /* Constant */
2
3 printf("ANSWER: %d\n", ANSWER);
```

ends up:

```
1 printf("ANSWER: %d\n", (42));
```

There is no replacement in string literals.

Conditional Replacements

`#if`, `#ifdef`, `#ifndef`, `#elif`, `#else`, `#endif`:

```
1 #ifdef WIN32
2 #include <windows.h>
3 #else
4 #include <unistd.h>
5 #endif
```

```
1 #if DEBUG >= 2
2 printf("debug, debug\n");
3 #endif
```

- Complex macros with parameters can be defined

```
1 #define NRELEMENTS(a) (sizeof(a) / sizeof(a[0]))
```

- Macros should be used with caution! There are a lot of risks and side effects

Risks


```
1  #define DOUBLE(a) a+a
2
3  int x = DOUBLE(5) * 3;
4  /*  x = 5 + 5 * 3 <=> 5 + (5 * 3)
5  => #define DOUBLE (a) ( (a) + (a) ) */
```

```
1  #define DOUBLE(a) ( (a) + (a) )
2
3  int x = 3;
4  int y = DOUBLE(++x);
5  /*  y = ( (++x) + (++x) ) */
```

Undefined Behaviour

The result of some operations is explicitly left undefined by the C standard:

- Division by zero
- Out of bounds array access
- Expressions with "strange" effects on state, e.g., `y = ( (++x) + (++x) )`
- ...

The compiler is free to handle these cases however it likes!

- Program might crash
- Or it might continue to work but with "wrong" results
- (Wrong from your point of view)

Tip: Enable all warnings (we will see how)